ORGNZ-08: Grail Segments

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 8 of 10 | **Created:** January 2026 | **Last Updated:** 11/07/2026

Overview

Grail segments are logical groupings of data that enable real-time filtering across massive datasets without creating thousands of individual rules. Segments bring business context to observability data and are consistently available across the Dynatrace platform.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:filter-segments:read`, `storage:logs:read`, `storage:entities:read`
Knowledge	Completed ORGNZ-07 (Advanced Permissions)
Data	At least 1 hour of log, span, and entity data

Table of Contents

1. Segments vs Buckets
2. Segment Use Cases
3. Segment Structure
4. Segment Limits

5. Segment Design Patterns
6. Leveraging OneAgent Enrichments
7. Entity Relationship Strategies
8. Testing Segments
9. Scoping Services and Smartscape
10. Segments and Access Control
11. Segment Design Checklist
12. Bucket-Based Segment Filtering

Learning Objectives

By the end of this notebook, you will:
- Understand segments vs buckets
- Design effective segment rules
- Use dynamic variables in segments
- Apply entity relationships for comprehensive filtering

Segments vs Buckets

Aspect	Buckets	Segments
Purpose	Physical data storage	Logical data filtering
Retention	Controls data retention	No impact on retention
Data types	One type per bucket	Multiple types per segment
Access control	Can be used with IAM	Data filtering only
Cost impact	Direct storage costs	No direct cost impact
Query scope	Storage-level partitioning	Query-time filtering

> **Key insight**: Buckets store data; segments filter it. Use buckets for retention and cost allocation, segments for dynamic data views.

> **Performance benefit**: Applying a segment also reduces the **scanned bytes** of your queries, compared with the same query run without a filter. A segment works by injecting filter conditions at query time, so it costs what the same filter written into the query costs: no more, and no less. Which filter it injects decides the saving. A segment on `dt.system.bucket` skips the other buckets entirely; a segment on a field such as `k8s.namespace.name` still reads every record in every bucket the query can reach. On a validation tenant (10/23/2026, a 15-minute window that scanned 731.8 MB unfiltered), a bucket filter for `easytrade` scanned 317.1 MB over 1.28M records, and a namespace filter returning the same rows scanned 419.4 MB over all 3.51M records. The saving matters most on high-volume buckets, where unfiltered queries might hit the default 500 GB scan limit (`scanLimitGBytes`).
>
> **Sources:** Segments in DQL queries (DT docs) — *"segments will directly impact how much data is scanned and returned for a given query"*; segments act *"injecting user-defined, preconfigured filter conditions."* **Derived:** a segment scans what its injected filter scans, so the bucket-versus-field difference is the measured one above.

Segment Use Cases

1. Team-Based Views

```
Segment: "Platform Team"
Includes: Infrastructure hosts, Kubernetes clusters, platform services
Benefit: Team sees only relevant data, faster troubleshooting
```

2. Application Isolation

```
Segment: "Checkout Application"
Includes: Checkout service, related databases, frontend components
Benefit: Complete application view across logs, traces, metrics
```

3. Environment Filtering

```
Segment: "Production Only"
Includes: Entities with environment=production tag
Benefit: Focused production monitoring without dev/test noise
```

Segment Structure

```
Segment:
  name: "segment-identifier"
  displayName: "Human Readable Name"
  description: "What this segment filters"

  includes:
    - type: Data (all types)       # one condition across signals and
Smartscape nodes
      filter: 'dt.host_group.id = "prod-*"'

    - type: Entities               # Smartscape nodes only (More &gt;
Entities)
      filter: 'type = "HOST" AND dt.host_group.id = "prod-*"'

  variables:
    - type: query                  # a DQL query; its first column is the
variable
      value: "smartscapeNodes \"HOST\" | fields dt.host_group.id | dedup
dt.host_group.id"
```

Filter conditions use the segment filter grammar (`=`, `in()`, `*` inside the value) — see Segment Limits. Dynatrace advises against legacy `dt.entity.*` attributes in include blocks; see Entity Relationship Strategies.

Key behavioral rules:

- **Include blocks are ORed together** — data matching *any* include is in scope; display order in the UI does not affect which data is included
- **One include per data/entity type** — you cannot add two log includes; use AND/OR within a single filter condition instead
- **Evaluated at query time** — filter conditions inject into every query that uses the segment; complex conditions increase query overhead. Preprocess fields via **OpenPipeline** to simplify conditions and improve performance

Segment Limits

Limit	Value	Notes
Segments per environment	10,000	Tenant-wide ceiling
Maximum includes (rules) per segment	20	Plan rule complexity accordingly
Include blocks per data/entity type	1	Cannot have multiple rules for same type
Expressions per filter condition	1 min / 10 max	Maximum conditions per filter
Values per variable	15,000	Maximum values in the variable dropdown; a 5 MB byte cap also applies
Selected variable values per segment	100	Classic entities
Additional include per classic entity type + relationship	1	One relationship hop only

Operators and Wildcards

Segment filter conditions offer only two operators: `=` **and** `in`. There is no separate `contains` operator — substring and suffix matching is expressed by placing a **wildcard** `*` **inside the value**. Conditions are written in the **filter-field grammar** of the segment editor (`key = value` , `key in (value1, value2)`), not in DQL: the DQL function form `in(field, {"a", "b"})` is a different language.

Matching	How to express it	Where it works
Equals	`field = "value"`	Everywhere
Set membership	`field in ("a", "b")`	Everywhere — exact values only; `*` inside the list is not a wildcard
Starts-with	`field = "prefix*"`	Signal data; classic `entity.name`
Contains	`field = "*text*"`	Signal data — not classic entity includes

Matching	How to express it	Where it works
Ends-with	`field = "*suffix"`	Signal data — not classic entity includes

> **Important**: on **classic entity** includes (legacy — see Entity Relationship Strategies), `entity.name` supports **starts-with only** (`entity.name = "payment*"`); contains and ends-with are not supported on any classic-entity property, and every other property is exact `equals` only. Signal-data includes (logs, spans, events, metrics) have no such restriction. Tag conditions are a separate case — a tag filter tests **membership in the tag set**, not a substring of a string field — so verify tag-based entity includes against your own tenant before relying on wildcard behavior there.
>
> An asterisk may also follow a variable name in a starts-with condition — for example `foo = $bar*` — which is what makes variable-driven segments practical.
>
> **Sources:** Segment limits (DT docs) — *"15,000 values in the variable dropdown (5 MB byte cap also applies)"*; *"Only operators `=` , and `in()` are supported."*; classic entities: *"starts-with, contains, and ends-with are not supported"*, *"entity.name supports starts-with"*, *"Other properties only support equals"*. Filter field (DT docs) — `in` *"matches one or more values in a list of values"*; *"Wildcards are not supported within the in () operator."*

Segment Design Patterns

Pattern 1: Team-Based Segment

All data owned by a team, keyed on a primary Grail tag set at the source:

Include	Filter
Data (all types)	`primary_tags.team = $primary_tags.team`

One `Data (all types)` include reaches logs, spans, events, metrics and Smartscape nodes that carry the tag. Add the variable and `Entities` include from Scoping Services and Smartscape if the team uses the segment in Services or Smartscape.

Pattern 2: Application-Based Segment

One application's services, logs and spans, scoped by its namespace. Because the segment is meant for **Services** and **Smartscape** too, it uses the three-part build rather than a `dt.entity.service` include:

Include	Filter
Data (all types)	`k8s.namespace.name = $k8s.namespace.name`
Variable	`metrics` query returning `k8s.namespace.name` with its `dt.smartscape.service` IDs
Entities	`type = "SERVICE" AND id in ($dt.smartscape.service)`

Pattern 3: Environment-Based Segment

Production workloads only, from an environment tag:

Include	Filter
Data (all types)	`primary_tags.environment = $primary_tags.environment`

The variable lists the environments users can pick. A host group that encodes the stage works the same way with a starts-with value, `dt.host_group.id = "prod-*"`.

Leveraging OneAgent Enrichments

OneAgent automatically enriches signals with metadata for precise filtering:

Enrichment	Source	Example Use
`dt.host_group.id`	Host group configuration	Filter by host group
`k8s.namespace.name`	Kubernetes namespace	Filter by namespace
`k8s.cluster.name`	Kubernetes cluster	Filter by cluster
`cloud.provider`	Cloud detection	Filter by AWS/Azure/GCP
`cloud.region`	Cloud region	Filter by region

Enrichment	Source	Example Use
`tags`	Entity tags	Filter by custom tags

Host Group Filtering Example

Include	Filter
Data (all types)	`dt.host_group.id = "prod-web-tier"`

`Data (all types)` is the recommended include for Smartscape nodes as well as signals, so the one condition covers logs, spans and the hosts themselves — no separate host-entity include.

> **Sources:** Filter Smartscape nodes with segments (DT docs) — *"you can filter monitored entities using the recommended Data (all types) include block or the type-specific include block for entities."*

Entity Relationship Strategies

> **Legacy only — don't build new segments on it.** Entity relationships in segments *"are only supported for backward compatibility with classic entities."* They apply to `dt.entity.*` attributes and *"don't work with Smartscape entities"*, and Dynatrace advises against `dt.entity.*` attributes in include blocks altogether; the segment editor shows a warning on existing ones.

For new segments, filter on Primary Grail Fields or primary Grail tags with a `Data (all types)` include. For services, use a variable to derive the service IDs (see Scoping Services and Smartscape), not traversal from hosts or process groups.

If an existing segment still relies on traversal, the rules are:

- Only **one relationship hop** is supported per include block
- Multiple **parallel** relationships from the same originating entity type are allowed

Replace it the next time you edit the segment.

> **Sources:** Include data in segments (DT docs) — as quoted, Use segments to scope Services and Smartscape apps (DT docs) — *"Do not use legacy dt.entity.* attributes in include blocks in your segments."*

Testing Segments

DQL: Segment Simulation

Simulate a host-group-based segment filter on log data:

```
// Filter logs by host group — simulates a segment filter condition on log
data.
// `dt.host_group.id` is a Primary Grail Field: Dynatrace enriches it on every
signal from a host
// that has a host group, so it is the canonical attribute to anchor a host-
group-based segment on.
// Coverage on derived records (problems) can be far lower than on logs
(ORGNZ-10 §3).
// The legacy `host.group` field is not populated on modern OneAgents and
returns zero rows.
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id)
| summarize count = count(), by:{dt.host_group.id}
| sort count desc
| limit 10
```

Scoping Services and Smartscape

A segment for the **Services** app and for **Smartscape** has to reach two data sources. The service list is built from service metrics (response time, throughput, failure rate), and the topology view is built from Smartscape nodes. In Dynatrace's words: *"A segment that covers only metrics filters the service list but leaves the topology view unfiltered."*

The documented build has three parts:

Part	What to enter	What it scopes
Data (all types) include	`k8s.namespace.name = $k8s.namespace.name`	The service list. Filtering on a primary Grail field or tag works because *"Dynatrace automatically carries that attribute through to the matching service metrics."*
Variable	A `metrics` query that returns each value with its `dt.smartscape.service` IDs (below)	Nothing by itself; it produces the secondary variable `$dt.smartscape.service`
Entities include (*More* > *Entities*)	`type = "SERVICE" AND id in ($dt.smartscape.service)`	Service nodes in Smartscape

Reference the field as `$<field>` rather than a literal, as the tutorial advises, so one segment covers every namespace (or region, host group or team).

Do not use `dt.entity.service`. The tutorial says so directly: *"Do not use dt.entity.service in your segments. Instead, use dt.smartscape.service."* Legacy `dt.entity.*` attributes *"might incorrectly filter the service list"*. The classic-entity relationships under **Classic entities (by type)** don't help either: use the variable to derive service IDs instead.

```
// Segment variable definition: services per Kubernetes namespace.
// Adapted from the Dynatrace Services-and-Smartscape segment tutorial (its
example uses aws.region).
// The isNotNull(dt.smartscape.service) guard is an addition: without it the
collected list
// also holds a null, from metrics that carry the field but no service.
// Live-verified 10/14/2026: 4 namespaces with 2-17 services each; the IDs
resolve as
// smartscapeNodes "SERVICE" records (6 of 6 checked). 0 bytes scanned.
metrics
| filter isNotNull(k8s.namespace.name) and isNotNull(dt.smartscape.service)
| fields k8s.namespace.name, dt.smartscape.service
| summarize {dt.smartscape.service = collectDistinct(dt.smartscape.service)},
by:{k8s.namespace.name}
| sort lower(k8s.namespace.name)
```

The first column becomes the primary variable `$k8s.namespace.name`, which is the value users pick. The second becomes the secondary variable `$dt.smartscape.service`, which

the `Entities` include reads.

Two things to plan around:

- **10,000 services.** The variable reads service IDs from service metrics, and the approach *"works reliably for up to 10,000 services."* Above that, the variable query *"may return an incomplete list, leaving some services unfiltered."* Split a broad segment into narrower ones, for example one per cluster or region.
- **Build all three parts, whatever the field.** For a team tag, the tutorial says `Data (all types)` scopes Services because *"primary_tags.* fields are propagated to service metrics"*, but Smartscape still needs the other two parts. For a host group, it says *"a host group is not automatically propagated to service metrics"* and tells you to follow every step. (ORGNZ-10 §3 records a tenant observation that disagrees with that sentence; the build is the same either way.)

Check the result in two places, as the tutorial does: **Services > Explorer (Early Access)** with *Filter by segments*, and **Smartscape > Service dependency graph** with the segment selector.

> **Sources:** Use segments to scope Services and Smartscape apps (DT docs) — published 10/07/2026; all quotes above, Filter Smartscape nodes with segments (DT docs) — the `Entities` include block, *More > Entities*.

Segments and Access Control

Important: Segments are for **data filtering only**, not access control.

Function	Segments	IAM Policies
Filter data in queries	Yes	No
Restrict user access	No	Yes
Enforce security boundaries	No	Yes
Dynamic data views	Yes	No

Segments are governed by existing access controls - users only see data they're authorized to access.

Segment Design Checklist

- [ ] Defined clear segment purpose
- [ ] Identified all data types to include
- [ ] Verified metadata consistency (tags, host groups)
- [ ] Planned rule structure (max 20 includes)
- [ ] No legacy `dt.entity.*` attributes or classic-entity relationships in include blocks
- [ ] Segments for Services or Smartscape have the variable and `Entities` include
- [ ] Tested filter conditions with DQL
- [ ] Documented segment usage
- [ ] Planned variable usage for flexibility

Bucket-Based Segment Filtering

Segments can filter by **`dt.system.bucket`** — making them useful for restricting dashboards and notebooks to specific storage buckets without modifying the query itself.

Static Bucket Segment

A segment that always targets a single bucket:

Field	Value
Data type	Logs
Filter condition	`dt.system.bucket = "default_logs"`

Dynamic Bucket Segment (with Variable)

Use a variable to make the bucket selection interactive at runtime:

```
Variable DQL:
fetch dt.system.buckets
| filter dt.system.table == "logs"
| fields bucket = name
| sort bucket asc
```

Then reference `$bucket` in the filter condition: `dt.system.bucket = $bucket`

This lets users pick a bucket from a dropdown in Dashboards or Notebooks without changing the underlying DQL query — useful for security reviews (pick the `audit_logs` bucket) or cost analysis (pick a specific team bucket).

> **Performance benefit:** a bucket segment skips the other buckets, so it scans what `bucket:{...}` scans. That is less than an unscoped query, and Log query consumption is billed per GiB scanned. It is no cheaper than writing the same bucket filter into the query yourself.
>
> **Sources:** Segment logs by bucket (DT docs) — *"Configure a segment to function as a convenient filter for logs by Grail buckets to optimize query performance and license consumption."* Log Management & Analytics - Query (DT docs) — *"It is calculated per gibibyte scanned (GiB scanned)."*

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-09**: Enterprise Patterns
- **ORGNZ-10**: Advanced Segment Definitions

References

- Grail Segments
- Filter field (DT docs)
- Smartscape topology and entities (DT docs)
- Filter Smartscape nodes with segments (DT docs)
- Include data in segments (DT docs)
- Use segments to scope Services and Smartscape apps (DT docs)
- Segment logs by bucket (DT docs)
- Segment Limits
- Configure custom filter segments — get started (DT docs)

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.